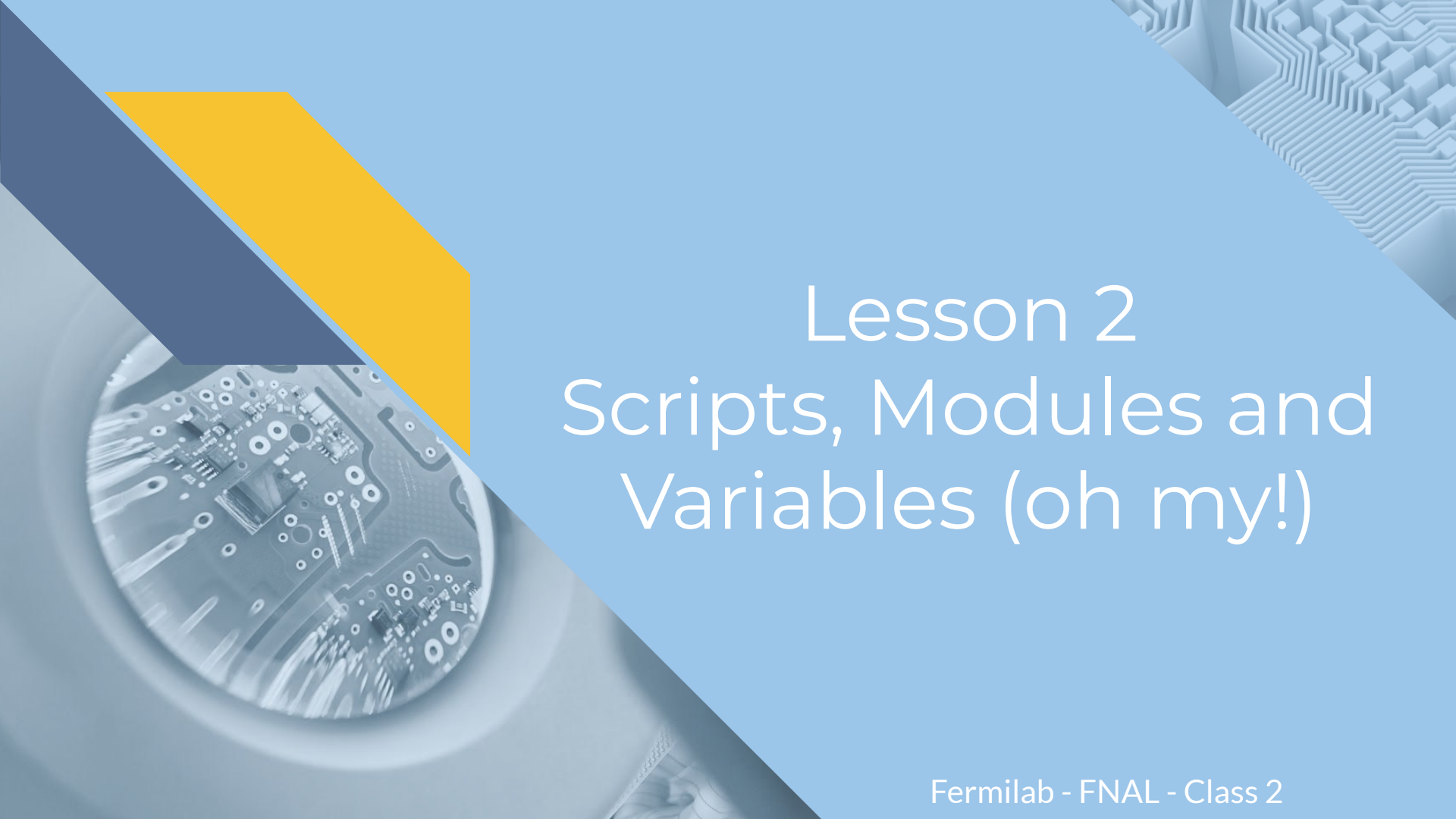
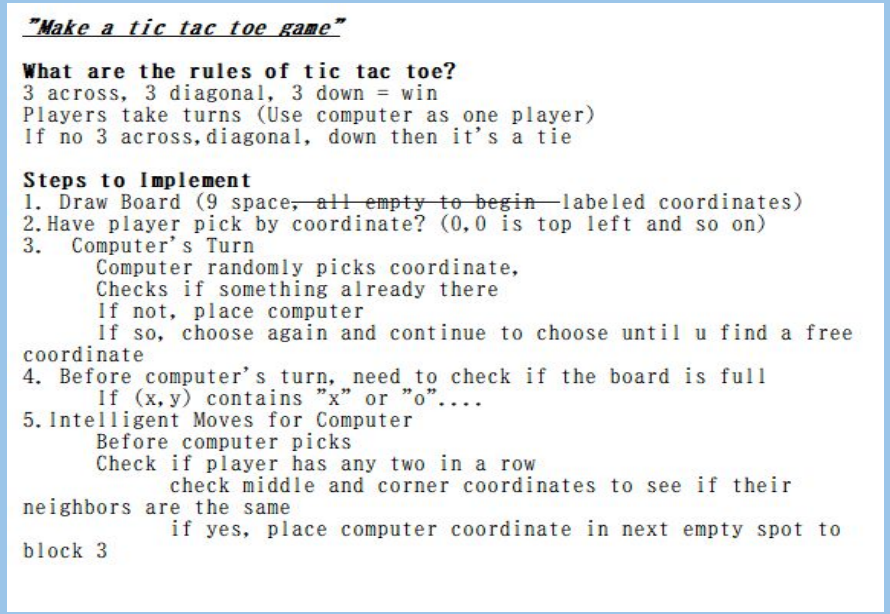


Lesson 2
Scripts, Modules and Variables (oh my!)

Fermilab - FNAL - Class 2

Now that we've experienced a little code and problem solving, an important topic to talk about is ***Pseudocode***

Pseudocode is a **high level** description of how you plan to implement your code in order to solve the problem.



"Make a tic tac toe game"

What are the rules of tic tac toe?
3 across, 3 diagonal, 3 down = win
Players take turns (Use computer as one player)
If no 3 across,diagonal, down then it's a tie

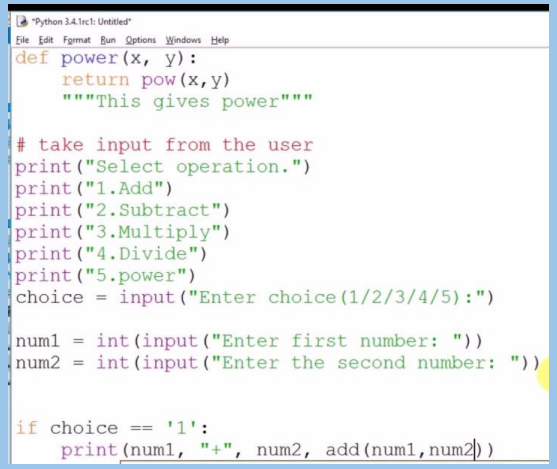
Steps to Implement
1. Draw Board (9 space~~, all empty to begin~~ labeled coordinates)
2.Have player pick by coordinate? (0,0 is top left and so on)
3. Computer's Turn
Computer randomly picks coordinate,
Checks if something already there
If not, place computer
If so, choose again and continue to choose until u find a free coordinate
4. Before computer's turn, need to check if the board is full
If (x,y) contains "x" or "o"....
5.Intelligent Moves for Computer
Before computer picks
Check if player has any two in a row
check middle and corner coordinates to see if their neighbors are the same
if yes, place computer coordinate in next empty spot to block 3

This brings us to ***Levels of Programming Languages.***

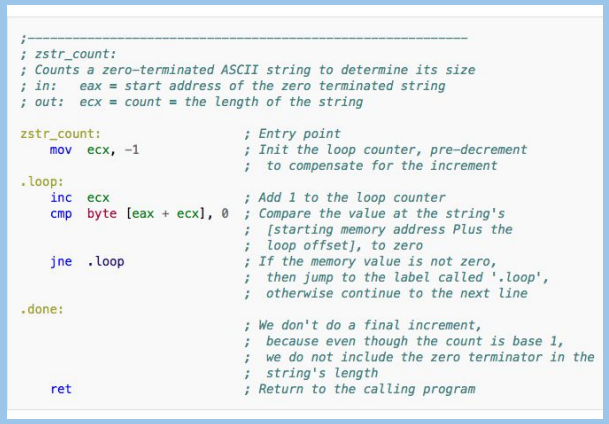
Programming Languages have three levels:

High, Mid/Assembly, Low/Machine Code

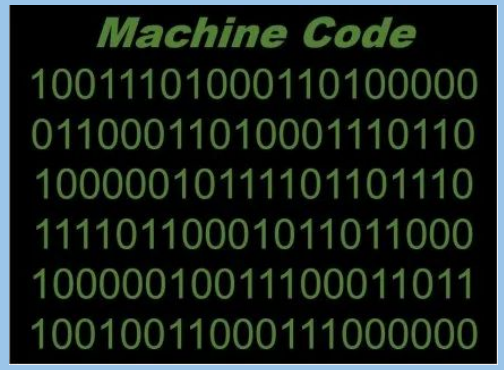
High Level languages are where the syntax is closer to human speech or mathematical formulas.



Mid Level/Assembly languages are where the syntax focus is more on mapping computer instructions, so it looks more like what a computer would make.



Low Level/Machine Code is 100% for the computer only (It' written in Binary!) so it's the format that computers understand.



Compilers and Interpreters - How the computer understands code!

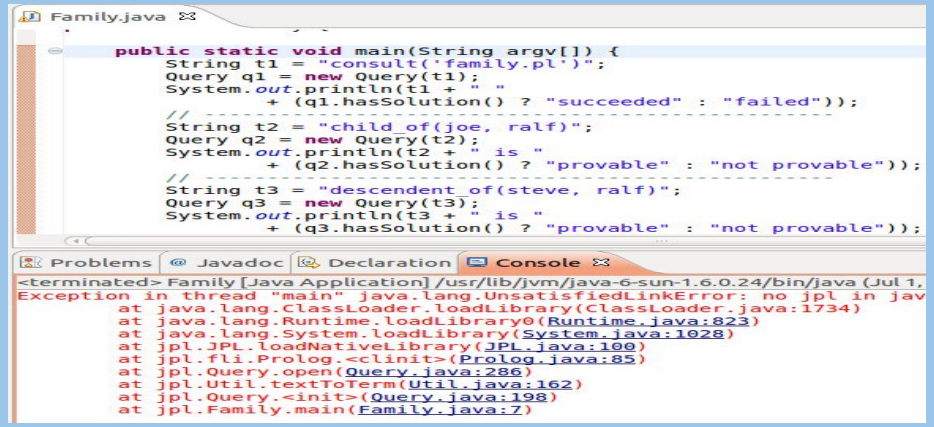
Compilers and Interpreters are types of translators for code. It takes it from a level easy for humans and translates it into binary for computers to understand.

Interpreters run the code a single line of code at a time. Once it hits an error, it stops scanning right there and returns the error, so it only gets one error at a time. ***Interpreters*** compile and run code at the same time, each time you want to run it. ***Python is interpreted.***

```
>>> sam_age = 15
>>> jim_age = 12
>>> if jim_age > sam_age:
        print("sam is older")
        else:

SyntaxError: invalid syntax (<pyshell#4>, line 3)
>>>
```

Compilers take the entire program/body of code and run it through. This means it's able to display all errors in the program at the same time, before running it. They are comparatively faster than ***Interpreters***. ***Compilers*** compile code once first and produce an executable to run.



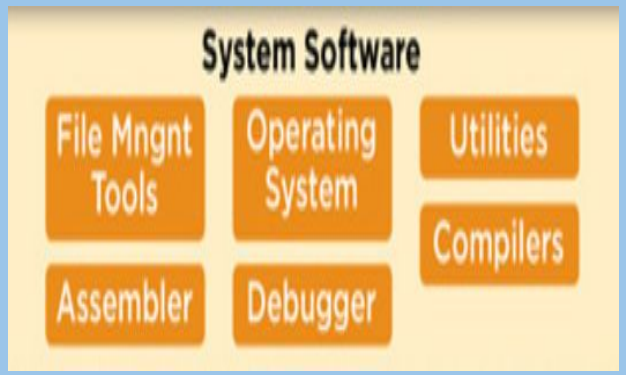
Software, Application, Program

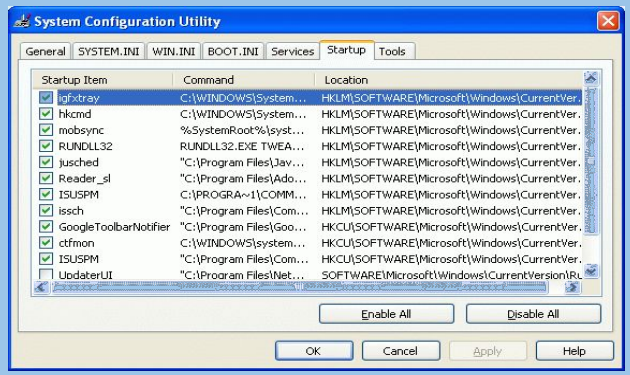
What kind of code are you making?

Software is the global term for all the components (programs) distinct to hardware that tell a device what to do and how to behave

Application is a type of software that does a **certain task**. Intended for a particular platform or device. Often needs user interaction to function.

Program is a (algorithm + data structures) sequence of instructions that comply with the rules of a specific programming language, written to perform a specified task with a computer





Note: All ***applications*** are ***programs*** but not all ***programs*** are ***applications*** because some programs do not **require** user.

File Structure in Python

Library, Standard Library, Package, Module

Module - single file of Python code that is meant to be imported

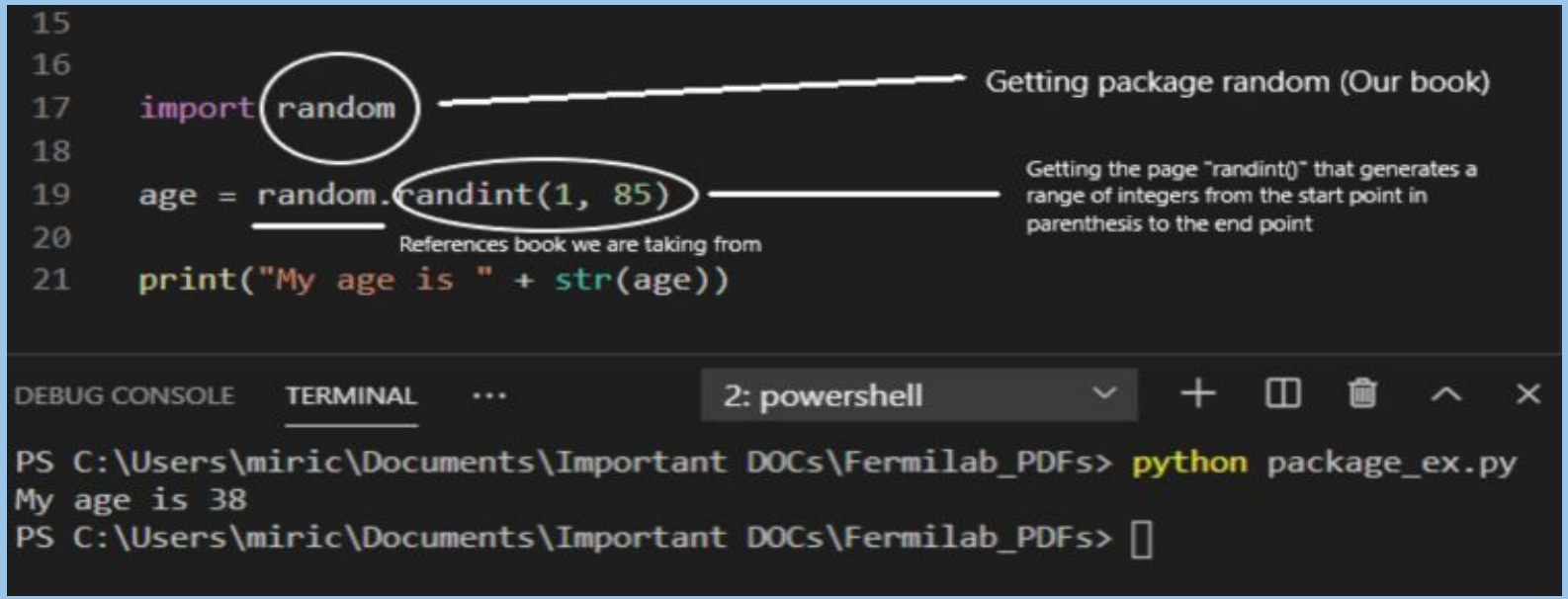
Package - collection of Python modules under a common namespace. In practice one is created by placing multiple python modules in a directory with a special __init__.py module (file)

Library - generic term for code that was designed with the aim of being usable by many applications. PYTHONPATH lists known libraries.

Standard library - collection of exact syntax, token and semantics of the Python language which comes bundled with the core Python distribution (>200 modules)

File Structure PT. 2

Think of ***Libraries/Standard Libraries*** as actual libraries and ***packages*** as books and ***modules*** as a particular chapter or page. Everything is stored in a library and to use it you have to check it out or "import" it. You import a ***package*** that has the entire book of everything you can use and you pick which page or "***module"*** to use. Here's an example!



File Structure PT. 3
Python Script

Import statements
to use other modules

Comments
Starting with #

Outside statements
Executed when running the script

```
#!/usr/bin/python
import os
import sys
import string
import optparse
...
from glideinwms.lib import condorExe
# Main function
def main(argv):
  feconfig=frontenvparse.FEConfig()  # FE configuration holder
  ... # parse arguments
  feconfig.config_optparse(argparser)
  (options, other_args) = argparser.parse_args(argv[1:])

  if len(other_args)<1:
    raise ValueError, "Missing glidein_name"
  glidein_name = other_args[0]
  if len(other_args)>=2:
    log_type=other_args[1]
  else:
    log_type="STARTD"
  ...
  return 0
# S T A R T U P
if __name__ == '__main__':
  try:
    sys.exit(main(sys.argv))
  except Exception, e:
    sys.stderr.write("ERROR: Exception msg %s\n"%str(e))
    sys.exit(9)
```

#! - shebang
Indicates which interpreter can run the script

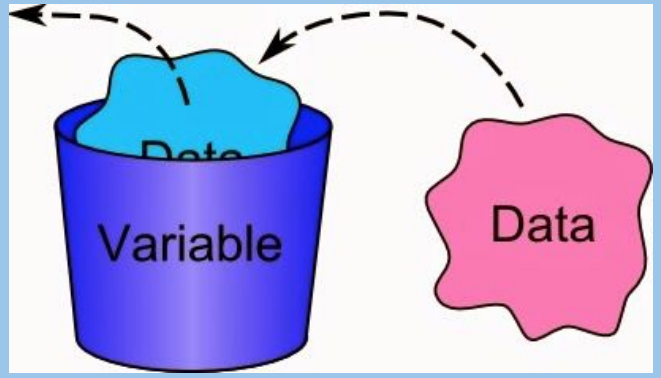
Block
Groups together a list of statements
4 spaces indentation!

Function
Gives a name to a block of code and hides variables

__main__
Value of __name__ when this is invoked as a script (as alternative could be imported)

Variables

Variables are used to store information to be referenced and manipulated in a computer program. Variables have a name, value, representation, a type



Think of ***variables*** as place holders for information.

Ex. “Charlie told me her birthday and I need to remember it so I can buy her a gift later so I’m going to save it in my phone notes under Charlie's Birthday.”

Code does the same thing! It saves some information under a name. It just looks a little different.

Ex. charlieBirthday = “01/23/1998”

Variables also have a ***scope*** which is the part of the program where you can access it.

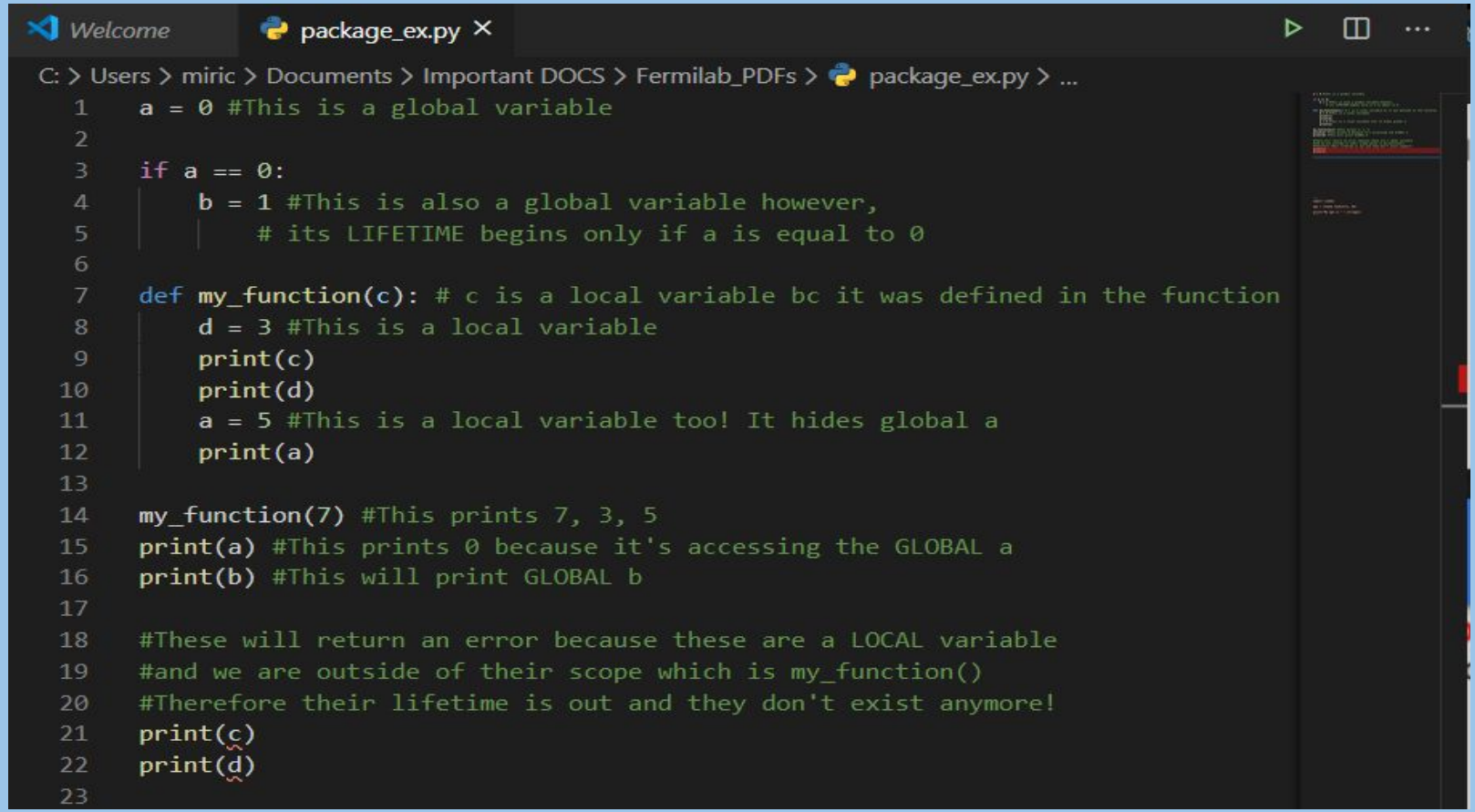
Scope is sectioned into two parts ***Global*** and ***Local***.

A ***Global Variable*** is available to be accessed throughout the entire program.

A ***Local Variable*** is accessible from the point at which it is defined until the end of the function, and exists for as long as the function is executing.

All variables also have a ***Lifetime*** or a duration for which a variable exists. Ex. A variable defined in a function only exists for the duration of the execution of that function.

Scope Example



Data Storage Side Note:

The further you get in code, the more complex the ways you can store and use data become!

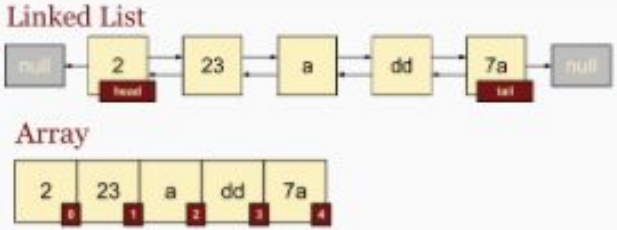
List

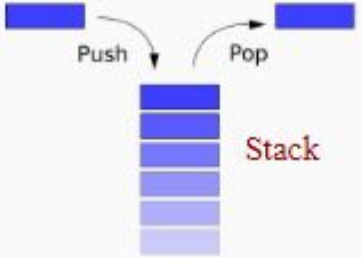
Array

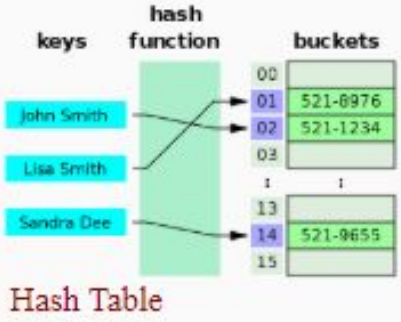
Stack

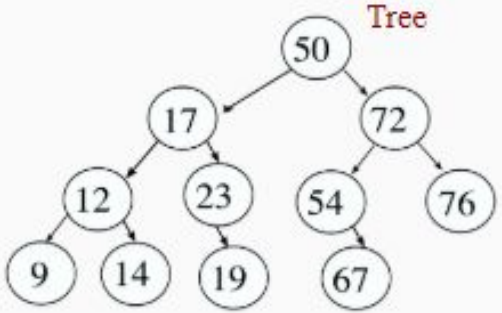
Hash Table (Map/Dictionary)

Tree









Knowing how to code may come handy...

Remember to choose a project for your presentation!

Here a creative example of someone using coding abilities to solve a problem

Project Mayhem writing some code to fight fake IRS phone scam (NOTE that the linked video contains some explicit language):

https://www.youtube.com/watch?v=EzedMdx6QG4

Alright! Let's Get Back to Coding!